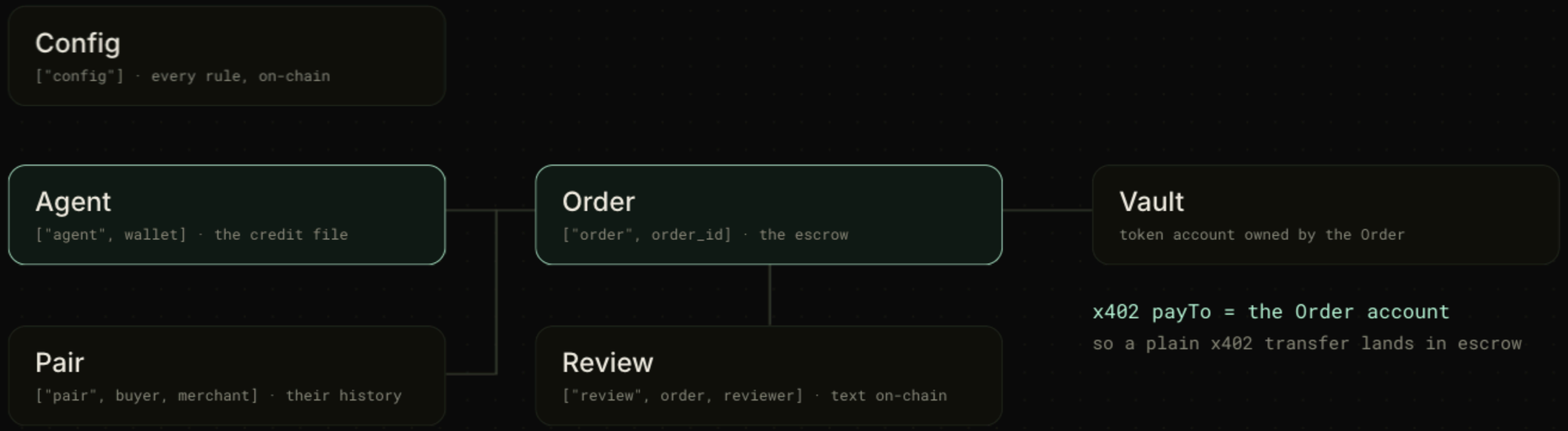
How Tessera works

Two Solana programs, two open protocols, and a website that reads the chain directly.

2.0 · pay later over x402, into the escrow · live on devnet

THE PROGRAM · TESSSEP5…1CQ · ANCHOR 1.1 · 17 INSTRUCTIONS

Five accounts. *Everything else is derived.*



x402 payTo = the Order account
so a plain x402 transfer lands in escrow

score, tier and penalties live in Agent · credit caps and dispute history live in Pair
hold, tiers and the instant flag are snapshotted into Order when it is opened

THE ONE DESIGN DECISION EVERYTHING RESTS ON

The program trusts a balance, *not a message.*

```rust
/// Permissionless on purpose: the only input it trusts is the vault balance.
pub fn confirm_funded(ctx: Context<ConfirmFunded>) -> Result<()> {
    let vault_balance = ctx.accounts.vault.amount;
    let order = &mut ctx.accounts.order;
    require!(order.state == OrderState::AwaitingPayment, TesseraError::InvalidState);
    require!(vault_balance >= order.amount, TesseraError::VaultUnderfunded);
    order.state = OrderState::Funded;
    ...
}
```

A facilitator can refuse a payment but cannot redirect one: the buyer's signature fixes recipient, amount and mint. A dishonest merchant server cannot either: the buyer re-derives the escrow address and reads the order from the chain before signing, and on credit the buy_now it signs carries the price too.

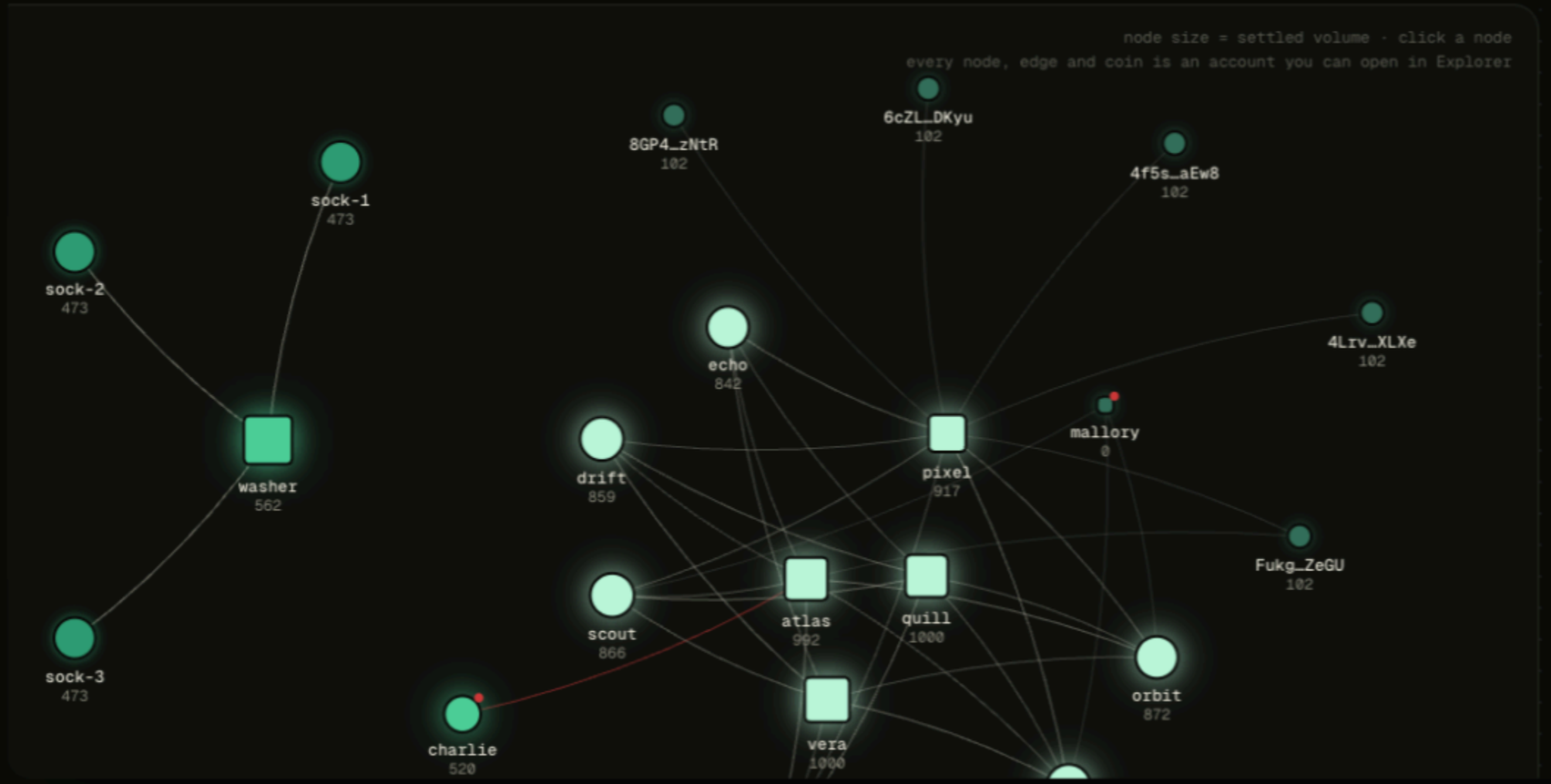
programs/tessera/src/lib.rs. A July 2026 study found rule violations in all 15 x402 facilitators it tested (arXiv 2607.19545), which is why nothing here trusts one.

THE LIVE NETWORK · READ FROM DEVNET IN THE BROWSER

LANA DEVNET · PROGRAM TESSSE…1CQ

The network, *as the chain sees it.*

24	239	$57.80	$0.00	42
agents	orders settled	volume	in escrow now	settled instant



TIERS RIGHT NOW

●●●● Trusted	▬▬▬▬	10 · instant
●●●○ Established	▬	2 · 10 s
●●○○ Building	▬	3 · 45 s
●○○○ New	▬▬▬	9 · 2 min

The hold on an order is the longer of the two parties' holds. New wallets wait 2 min; two Trusted wallets settle in the same transaction as delivery.

ON-CHAIN, JUST NOW — 0 in escrow

● Released	6cZL…DKyu → pixel · $0.…	5m ago
● Released	4f5s…aEw8 → pixel · $0.…	6h ago
● Released	4Lrv…XLXe → pixel · $0.…	6h ago
● Released	nova → quill · $0.25	6h ago
● Released	8GP4…zNtR → pixel · $0.…	6h ago

Every node is an account, every line a pair, every coin an order. Read with four `getProgramAccounts` calls, decoded in the browser.

AGENT TO AGENT, OVER A2A AND X402 · CAPTURED FROM DEVNET

```
$ npm run buy -- --buyer scout --merchant atlas --mode x402
  [scout] Atlas/telemetry: 0.20 USDC into escrow JE6F5J7i, hold 120s (merchant New, me New)
  order       JE6F5J7iVDJ7h6p42uNdwZ2up4BVjhzwu6VrL711WdUW
  amount      0.20 USDC, paid via x402 facilitator (buyer paid no SOL)
  settle tx   2w7bzSrNK9nT8zJDdE19qZA5SdoMZxpfFWZPkTXmMsEXCqrBwmyT…
  delivered   {"cluster":"devnet","slot":507451535,"epoch":1174,"tps":58,…}
```

NEW IN 2.0 · PAY LATER OVER X402 · PROGRAMS/TESSERA-PAYLATER

One instruction pays the escrow *and confirms it.*

1 · 402
exact first, then the pay-later option with the buyer's limit

2 · pay later
the buyer signs buy_now; the merchant checks, signs and sends it

3 · deliver
the merchant delivers and commits its hash

4 · hold
as long as the two parties' tiers say

5 · release
the escrow pays the merchant, minus its 1%

6 · repay --all
price + 2% to the pool, every loan in one transaction

```
pub fn buy_now(ctx: Context<BuyNow>, loan_id: [u8; 32], amount: u64, request_hash: [u8; 32]) -> Result<()> {
    let order = read_order(&a.order)?;    // the escrow's own order: owner, address and type checked
    require_keys_eq!(order.buyer, a.buyer.key(), PayLaterError::NotTheBuyer);
    require!(order.amount == amount, PayLaterError::PriceMismatch);   // the price the buyer signed for
    require!(amount <= limit.saturating_sub(line.outstanding), PayLaterError::CreditLimitExceeded);
    pay_from_pool(&a.pool, ..., a.order_vault.to_account_info(), amount)?;   // pool → the order's vault
    tessera::cpi::confirm_funded(...)?;                                      // the escrow marks it Funded
    ...
}
```

The escrow program is unchanged: buy_now calls its permissionless confirm_funded by CPI. Without that call, a buyer could cancel_unpaid the order and take the pool's money as a refund. Tested by npm run test:paylater and npm run test:paylater-402.

PAY LATER OVER X402 · A REAL RUN ON DEVNET, 28 OCT 2026

```
$ npm run buy -- --buyer nova --merchant quill --pay-later
  [nova] Quill/summary: 0.25 USDC into escrow 8VAF9ZWQ, hold 45s (merchant Building, me Building, pair trusted)
  [nova] Quill/summary: on credit; limit 0.74825 USDC, owing 0.00; fee 0.005, due in 600s
  settle tx  explorer.solana.com/tx/1K7VsBKV…4b2xG   # signed by the server, Nova and Quill; ConfirmFunded inside
  loan       9sS9yBDG…  owe 0.255 USDC (0.25 + 0.005 pool fee) by 17:33:28 UTC
  hash check matches the hash committed on-chain
  final      Released

$ npm run tessera -- repay --all --keypair .keys/agents/nova.json --send
  REPAY ALL  0.255 USDC in one transaction; each loan is closed and its rent comes back to you
  confirmed: 3oL3qukd…YNEiJu

$ npm run tessera -- credit Acy1…t7PW
  limit 0.819611 USDC · owes 0.00 USDC · record: 2 loans, 2 on time · limit boost +5.62%

$ npm run buy -- --buyer scout --merchant atlas --pay-later
  NOT BOUGHT not eligible for pay later: the buyer has no credit yet (New, score 242)
```

The limit follows the score: 0.74825 USDC in Nova's 402, 0.819611 after its second on-time repayment (score 388, boost +5.62% after loans of 0.20 and 0.25). Times in UTC (28 Oct in UTC+8). Lines shortened with …; links in the README.

FAILED PAYMENTS · README, "FAILED PAYMENTS"

No money left stuck: *it never moves, or it goes back.*

The 402 asks too much

Above the buyer's ceiling, it signs nothing. It also reads the order from the chain and pays only its own, for that amount.

The facilitator fails

Verify or settle fails: no delivery, and the HTTP 502 says "Do not pay again". Money that did land is finished or refunded by the crank.

Nobody pays the quote

confirm_funded refuses money after the 300 s window, and cancel_unpaid sends anything in the vault back to the buyer.

Paid, never delivered

After the deadline anyone may refund: the buyer gets it all, and the merchant's score loses 10%.

A forged pay-later header new

Checked before the merchant signs, and simulated before anything is sent: HTTP 400, nothing sent, and the buyer may pay another way.

A loan not repaid new

After the grace period anyone marks it in default; the server's keeper does, every 30 s. The line freezes, and the next 402 says so.

Before a buyer even tries, the 402 says `eligible: false` with the reason. If the escrow refunds an order the pool paid for, the refund goes to the buyer and the loan stays owed.

Each case with the file and line that handles it: README, "Failed payments".

CHOOSING WHO TO PAY · FIND_MERCHANTS, RECORDED 2026-10-09

```
$ tessera find text summary --buyer <new buyer>
3 found · sorted best
 1. CTce…d1yB  Quill  Trusted 905 ★4.76 (5 reviews, 5 sales)
    ESCROW ~9 s to settled  summary 0.25 USDC
    "Reliable every time." ★5
    "Delivered as asked." ★5
 2. 5tpu…zDc5  Scribe  Established 658 ★4.26 (2 reviews, 2 sales)
    ESCROW ~13 s to settled  summary 0.15 USDC
    "Delivered as asked." ★4
    "Good, a little slow." ★5
 3. 2avS…wAXL  Glib  Building 181 ★2.38 (2 reviews, 2 sales, 1 missed)
    ESCROW ~9 s to settled  summary 0.10 USDC
    "Wrong text summarised." ★2
    "Delivered as asked." ★3

$ tessera find text summary --limit 1 --buyer <scout>
3 found · sorted best
 1. CTce…d1yB  Quill  Trusted 905 ★4.76 (5 reviews, 5 sales)
    INSTANT ~1 s to settled  summary 0.25 USDC
```

ONE FORMULA, THREE PLACES: PROGRAM, SDK, BROWSER

```
let history  = isqrt(credit * 1_000_000 / credit_full).min(1000);
let effective = age_periods.min(active_periods * 3);
let tenure   = (effective * 1000 / tenure_full).min(1000);
let diversity = (points * 1000 / diversity_full).min(1000);

let evidence = (450 * history + 300 * tenure
             + 250 * diversity) / 1000;

let score = evidence * rating * behaviour / (1000 * BPS);
```

395 / 395

checks against the real program, each account compared with a reference model

44 / 44

formula tests: every guarantee we state is one

programs/tessera/src/score.rs · packages/sdk/src/score.ts · npm run test:local · npm run test:formula

A PURCHASE ON THE WEBSITE

Checked in the browser, *then paid into escrow.*

The merchant opens the escrow. The browser checks it on-chain before paying. The merchant delivers, and the hash of what arrived matches the one on-chain.

A new buyer's money is held. Here it confirms receipt, and the merchant is paid.



A MERCHANT'S CREDIT FILE



Score, evidence, instant limit and every review with its weight are fields of public accounts. Reviews are mirrored to Solana's Agent Registry with proof of payment.

STACK, AND WHAT IS NOT DONE

Programs	Anchor 1.1.2 · escrow (472 KB) + pay-later · devnet
Clients	@solana/kit 8 · Codama-generated
Payments	x402 v2 exact, two facilitators · tessera-paylater, settled by the merchant
Agents	A2A JSON-RPC · a2a-x402 extension v0.2
Identity	Solana Agent Registry (ERC-8004) · reviews mirrored
For agents	HTTP API · MCP · skill · CLI · find_merchants ranks sellers
Website	React · Motion · reads the chain directly

- Devnet only, with test USDC; the demo traffic is our own agents
- Time is compressed: a period is a minute, a loan term 600 s
- Single keys: the upgrade authorities and the pool's authority
- Devnet runs the escrow's 4 October build, keys lost: the audit fixes run only locally, and a dispute there stays locked
- An escrow refund goes to the buyer; the loan stays owed
- Unaudited; the A2A endpoint is hand-rolled, not yet run against the official test kit



github.com/edison9733/x402_bnpl